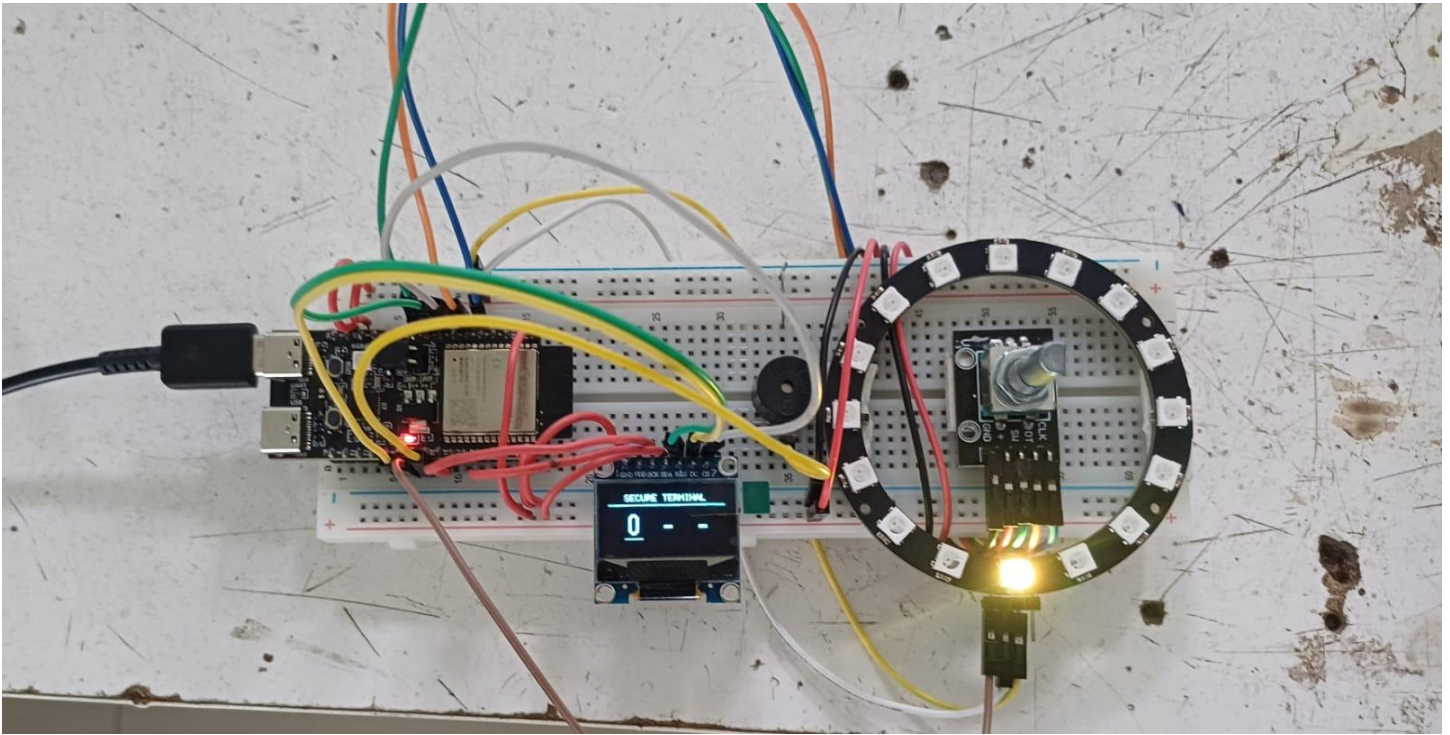


Project Safecracker Playbook

I. Challenge

Develop a digital combination lock vault interface that merges physical rotational inputs with multi-tiered security authentication layers. The system must evaluate phase-shifted quadrature signals via interrupt service routines (ISRs), implement bitwise state-tracking to eliminate mechanical contact bounce, drive a 1:1 hardware-mapped RGB LED ring, manage a multi-mode finite state machine (LOCKED, SUCCESS, ALARM), and render real-time UI feedback on an SPI-driven OLED display.



II. Guide

A. Concept

i. Underlying Engineering Principles & Reference Links

- Quadrature Decoding & Phase Shifts:** Quadrature rotary encoders output square waves across two channel pins (Channel A and Channel B). Rotating the shaft produces a 90° phase shift between channels, yielding a binary gray-code sequence (00 → 01 → 11 → 10). Evaluating which channel leads determines rotation direction.
 - Learn More: [Introduction to Rotary Encoders](#)
- ISR Bitwise State-Tracking (Hardware Debouncing):** Mechanical switch contacts inside rotary encoders suffer from bounce noise spikes. Instead of using blocking `delay()` functions, an Interrupt Service Routine (ISR) shifts previous and current 2-bit channel states into a 4-bit nibble (`movementState`). The counter increments or decrements only when valid physical state

sequences are matched, filtering out noise.

- *Learn More:* [Interrupt Service Routines Best Practices](#)

3. **Finite State Machines (FSM) in Security Architecture:** Multi-mode security systems rely on rigid operational states (LOCKED, SUCCESS, ALARM). Transition conditions evaluate user input strings against a hardcoded passcode array (PASSCODE[3]), incrementing failure counters or shifting to lockout modes upon reaching limit thresholds.

- *Learn More:* [Finite State Machines](#)

4. **1:1 Hardware-to-Visual Index Mapping:** Mapping physical rotational encoder positions (0 – 15) directly to physical indexes on a 16-element addressable LED ring provides instantaneous visual positioning feedback to the operator.

- *Learn More:* [Adafruit – NeoPixel Driver Math & Index Mapping](#)

ii. Real-World Example

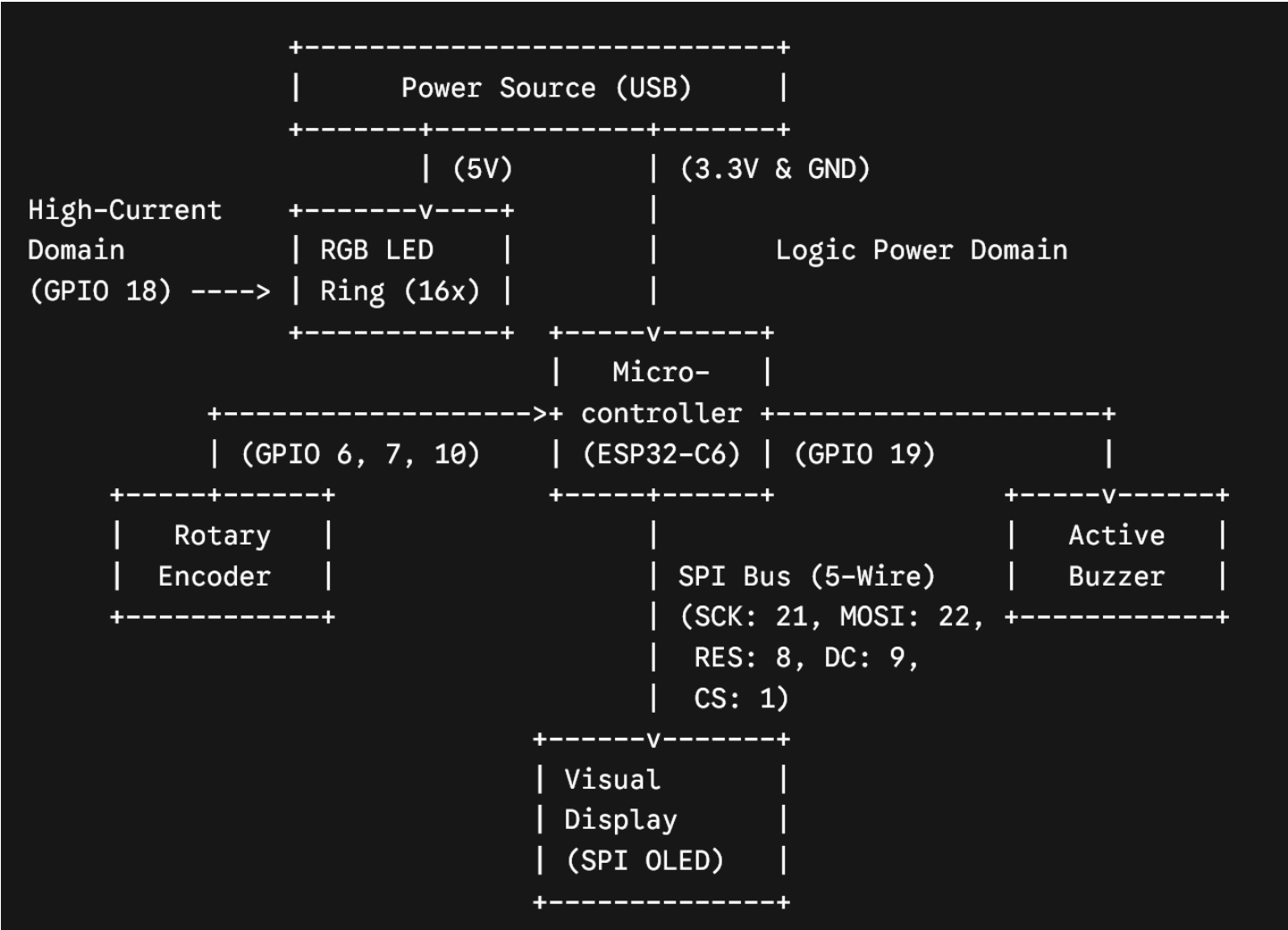
Commercial bank vault combination dials, digital safe keypads, industrial rotary selector switches, and automotive infotainment control knobs (e.g., BMW iDrive or Audi MMI systems). These applications rely on quadrature encoding and FSM security logic for tactile user interaction.



B. Components

- **Microcontroller:** Espressif ESP32-C6-DevKitC-1 (32-bit RISC-V SoC)
- **Input Peripheral:** Quadrature Rotary Encoder (with integrated push-button switch)
- **Visual Peripherals:** 16-LED RGB Ring Module (WS2812B) & 0.96-inch SPI OLED Display (SSD1306 driver)
- **Audio Transducer:** Active Buzzer Module
- **Interconnect Base:** Solderless breadboard and jumper wires

Block Diagram

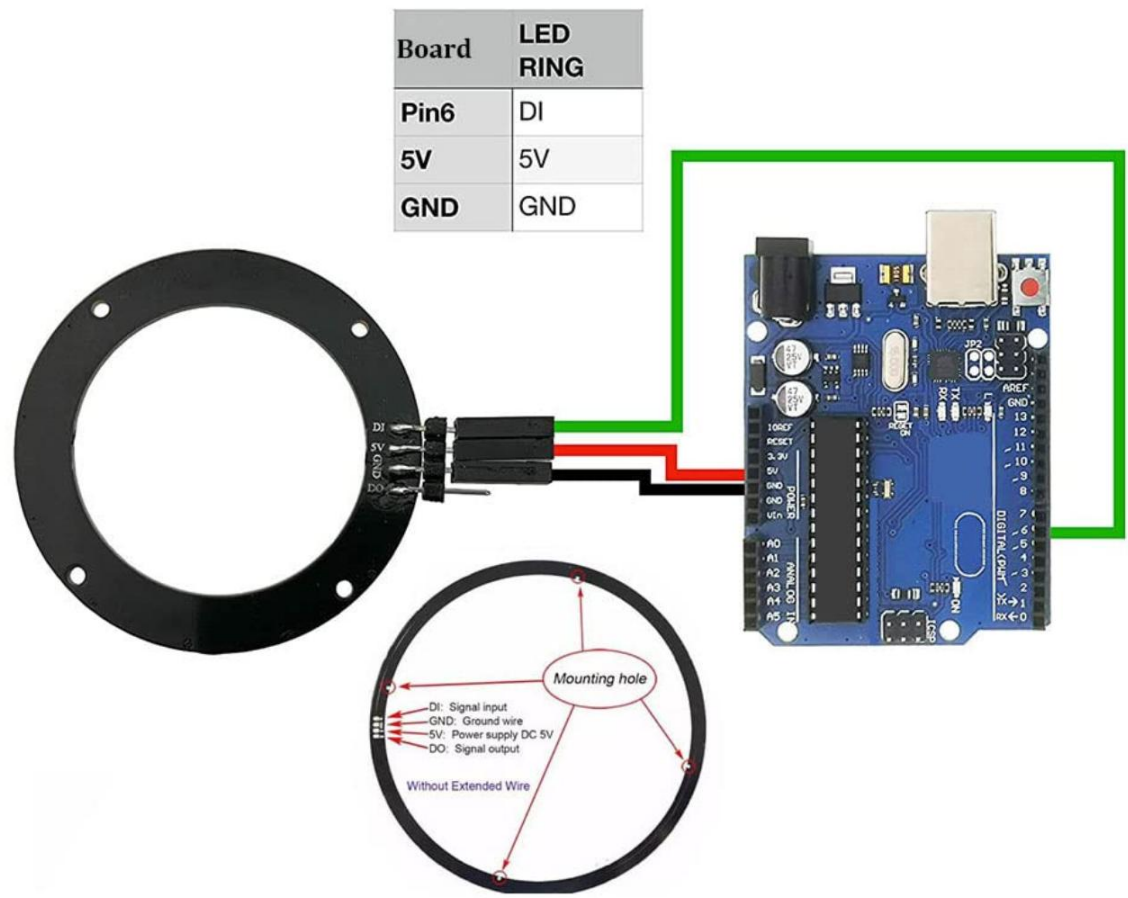


C. Connections

Module Pin	ESP32-C6 DevKit Pin	Signal Type	Power Domain / Function Description
Encoder CLK	GPIO 6	Interrupt (Phase A)	3V3 Logic Input
Encoder DT	GPIO 7	Input (Phase B)	3V3 Logic Input
Encoder SW	GPIO 10	Input (Button)	3V3 Logic Input (Internal Pull-Up)
Encoder VCC	3.3V	Logic Power	3V3 Logic Power Rail
OLED SCK	GPIO 21	SPI Clock	3V3 Logic Output
OLED MOSI	GPIO 22	SPI Data	3V3 Logic Output

Module Pin	ESP32-C6 DevKit Pin	Signal Type	Power Domain / Function Description
OLED RES	GPIO 8	Reset Flag	3V3 Logic Output
OLED DC	GPIO 9	Data/Command	3V3 Logic Output
OLED CS	GPIO 1	Chip Select	3V3 Logic Output
OLED VCC	3.3V	Logic Power	3V3 Logic Power Rail
RGB Ring DI	GPIO 18	High-Speed Data	5V High-Current Domain Signal
RGB Ring VCC	5V / VBUS	Direct Power	5V High-Current Rail (Direct USB)
Buzzer I/O	GPIO 19	Digital Trigger	3V3 Logic Output (Active HIGH)
GND (All)	GND	Common Reference	System Ground Plane

For those getting stuck on the pinout for the RGB Ring, here is an example diagram on how it is connected to an Arduino UNO Development Board:



D. Code

```
1. C++
2. #include <Arduino.h>
3. #include <Wire.h>
4. #include <SPI.h>
5. #include <U8g2lib.h>
6. #include <Adafruit_NeoPixel.h>
7.
8. // --- Pin Definitions ---
9. #define SPI_OLED_SCK 21
10. #define SPI_OLED_SDA 22
11. #define SPI_OLED_CS 1
12. #define SPI_OLED_RES 8
13. #define SPI_OLED_DC 9
14.
15. #define ENCODER_CLK 6
16. #define ENCODER_DT 7
17. #define ENCODER_SW 10
18.
19. #define LED_RING_PIN 18
20. #define BUZZER_PIN 19
21. #define NUM_LEDS 16
22.
23. // --- Object Initialization ---
24. U8G2_SSD1306_128X64_NONAME_F_4W_HW_SPI oled(U8G2_R0, SPI_OLED_CS, SPI_OLED_DC, SPI_OLED_RES);
25. Adafruit_NeoPixel ring(NUM_LEDS, LED_RING_PIN, NEO_GRB + NEO_KHZ800);
26.
27. // --- Security State Machine ---
28. enum SystemState { LOCKED, SUCCESS, ALARM };
29. SystemState currentState = LOCKED;
30.
31. const int PASSCODE[3] = {9, 1, 15};
32. int userInput[3] = {0, 0, 0};
33. int currentDigitIndex = 0;
34. int failedAttempts = 0;
35.
36. // --- Volatile Encoder Tracking ---
37. volatile int encoderValue = 0;
38. volatile uint8_t previousPhase = 0;
39. int lastEncoderValue = 0;
40.
41. // --- Advanced State-Tracking Interrupt (Bounce-Proof) ---
42. void IRAM_ATTR readEncoder() {
43.     uint8_t currentA = digitalRead(ENCODER_CLK);
44.     uint8_t currentB = digitalRead(ENCODER_DT);
45.
46.     uint8_t currentPhase = (currentA << 1) | currentB;
47.     uint8_t movementState = (previousPhase << 2) | currentPhase;
48.
49.     // Exact clockwise physical sequence
50.     if (movementState == 0b1101 || movementState == 0b0100 || movementState == 0b0010 || movementState ==
0b1011) {
51.         encoderValue++;
52.         if (encoderValue > 15) encoderValue = 0;
53.     }
54.     // Exact counter-clockwise physical sequence
55.     else if (movementState == 0b1110 || movementState == 0b0111 || movementState == 0b0001 || movementState ==
0b1000) {
56.         encoderValue--;
57.         if (encoderValue < 0) encoderValue = 15;
58.     }
59.
60.     previousPhase = currentPhase;
61. }
62.
63. // --- Helper Functions ---
64. void setRingColor(uint8_t r, uint8_t g, uint8_t b) {
65.     for(int i = 0; i < NUM_LEDS; i++) {
66.         ring.setPixelColor(i, ring.Color(r, g, b));
67.     }
```

```

68.   ring.show();
69. }
70.
71. void soundChirp() {
72.   digitalWrite(BUZZER_PIN, HIGH);
73.   delay(40);
74.   digitalWrite(BUZZER_PIN, LOW);
75. }
76.
77. void setup() {
78.   Serial.begin(115200);
79.
80.   pinMode(BUZZER_PIN, OUTPUT);
81.   pinMode(ENCODER_SW, INPUT_PULLUP);
82.   pinMode(ENCODER_CLK, INPUT_PULLUP);
83.   pinMode(ENCODER_DT, INPUT_PULLUP);
84.
85.   attachInterrupt(digitalPinToInterrupt(ENCODER_CLK), readEncoder, CHANGE);
86.   attachInterrupt(digitalPinToInterrupt(ENCODER_DT), readEncoder, CHANGE);
87.
88.   SPI.begin(SPI_OLED_SCK, -1, SPI_OLED_SDA, SPI_OLED_CS);
89.   oled.begin();
90.
91.   ring.begin();
92.   ring.setBrightness(40);
93.   setRingColor(0, 0, 0);
94. }
95.
96. void loop() {
97.   unsigned long currentMillis = millis();
98.
99.   switch (currentState) {
100.
101.     case LOCKED: {
102.       userInput[currentDigitIndex] = encoderValue;
103.
104.       // Visual feedback: Direct 1:1 hardware mapping
105.       ring.clear();
106.       ring.setPixelColor(encoderValue, ring.Color(255, 200, 0));
107.       ring.show();
108.
109.       // Audio feedback
110.       if (encoderValue != lastEncoderValue) {
111.         soundChirp();
112.         lastEncoderValue = encoderValue;
113.       }
114.
115.       // Input Confirmation (Switch Pressed)
116.       if (digitalRead(ENCODER_SW) == LOW) {
117.         delay(50);
118.         while(digitalRead(ENCODER_SW) == LOW);
119.
120.         soundChirp(); delay(50); soundChirp();
121.
122.         currentDigitIndex++;
123.         encoderValue = 0;
124.
125.         // Verification phase
126.         if (currentDigitIndex >= 3) {
127.           if (userInput[0] == PASSCODE[0] && userInput[1] == PASSCODE[1] && userInput[2] == PASSCODE[2]) {
128.             currentState = SUCCESS;
129.           } else {
130.             failedAttempts++;
131.             setRingColor(255, 0, 0);
132.             digitalWrite(BUZZER_PIN, HIGH);
133.             delay(500);
134.             digitalWrite(BUZZER_PIN, LOW);
135.
136.             if (failedAttempts >= 3) {
137.               currentState = ALARM;
138.             } else {
139.               currentDigitIndex = 0;

```



```

140.         setRingColor(0, 0, 0);
141.     }
142. }
143. }
144. }
145.
146. // Render UI Dashboard
147. oled.clearBuffer();
148. oled.setFont(u8g2_font_6x10_tf);
149. oled.setCursor(18, 12); oled.print("SECURE TERMINAL");
150. oled.drawLine(0, 16, 128);
151.
152. oled.setFont(u8g2_font_logisoso22_tr);
153.
154. // Slot 1
155. if (currentDigitIndex > 0) oled.drawStr(20, 48, "*");
156. else if (currentDigitIndex == 0) { oled.setCursor(20, 48); oled.print(userInput[0]); }
157. else oled.drawStr(20, 48, "-");
158.
159. // Slot 2
160. if (currentDigitIndex > 1) oled.drawStr(58, 48, "*");
161. else if (currentDigitIndex == 1) { oled.setCursor(58, 48); oled.print(userInput[1]); }
162. else oled.drawStr(58, 48, "-");
163.
164. // Slot 3
165. if (currentDigitIndex > 2) oled.drawStr(96, 48, "*");
166. else if (currentDigitIndex == 2) { oled.setCursor(96, 48); oled.print(userInput[2]); }
167. else oled.drawStr(96, 48, "-");
168.
169. // Draw selector underline
170. if (currentDigitIndex == 0) oled.drawLine(18, 52, 20);
171. if (currentDigitIndex == 1) oled.drawLine(56, 52, 20);
172. if (currentDigitIndex == 2) oled.drawLine(94, 52, 20);
173.
174. oled.sendBuffer();
175.
176. // Diagnostic Telemetry Stream
177. Serial.print("State:"); Serial.print(currentState); Serial.print(",");
178. Serial.print("Encoder:"); Serial.print(encoderValue); Serial.print(",");
179. Serial.print("Slot:"); Serial.print(currentDigitIndex); Serial.print(",");
180. Serial.print("Failures:"); Serial.println(failedAttempts);
181. break;
182. }
183.
184. case SUCCESS: {
185.     setRingColor(0, 255, 0);
186.
187.     oled.clearBuffer();
188.     oled.setFont(u8g2_font_9x15B_tf);
189.     oled.setCursor(22, 38);
190.     oled.print("UNLOCKED");
191.     oled.sendBuffer();
192.
193.     Serial.print("State:"); Serial.println("SUCCESS_UNLOCKED");
194.     break;
195. }
196.
197. case ALARM: {
198.     if ((currentMillis / 200) % 2 == 0) {
199.         setRingColor(255, 0, 0);
200.         digitalWrite(BUZZER_PIN, HIGH);
201.     } else {
202.         setRingColor(0, 0, 0);
203.         digitalWrite(BUZZER_PIN, LOW);
204.     }
205.
206.     oled.clearBuffer();
207.     oled.setFont(u8g2_font_9x15B_tf);
208.     oled.setCursor(30, 38);
209.     oled.print("LOCKDOWN");
210.     oled.sendBuffer();
211.

```

```
212.         Serial.print("State:"); Serial.println("ALARM_LOCKDOWN");
213.         break;
214.     }
215. }
216. }
217.
```

Student Tip: Open the **Serial Monitor** (Ctrl+Shift+M) at **115200 baud** to trace state changes, track live encoder values, and monitor failed authentication attempts.

To inspect encoder positioning stability, open the **Serial Plotter** (Ctrl+Shift+L). Plotting Encoder against Slot and Failures lets you visually verify that bitwise ISR debouncing eliminates spurious counter jumps during rapid dial rotations.

III. Play & Share

- **Dynamic Passcode Generation / Master Key Reset:** Add a feature where holding the rotary encoder button down for 3 seconds while in the SUCCESS state prompts the user to dial in and save a brand-new 3-digit combination stored in non-volatile storage (Preferences.h).
- **Time-Based Duress Lockout Expire:** Modify the ALARM state using millis() so that instead of requiring a hard system power reboot, the vault undergoes a strict 60-second lockout penalty timer shown on the OLED screen before resetting back to LOCKED.
- **Tactile Haptic Click Synthesis:** Upgrade the active buzzer to a passive piezo transducer driven by ledcWrite(). Synthesize pitch clicks (1500 Hz tone for 10 ms) on encoder steps, and play distinct rising or falling musical arpeggios for successful entries or lockouts!

Use this section in case of parts variation

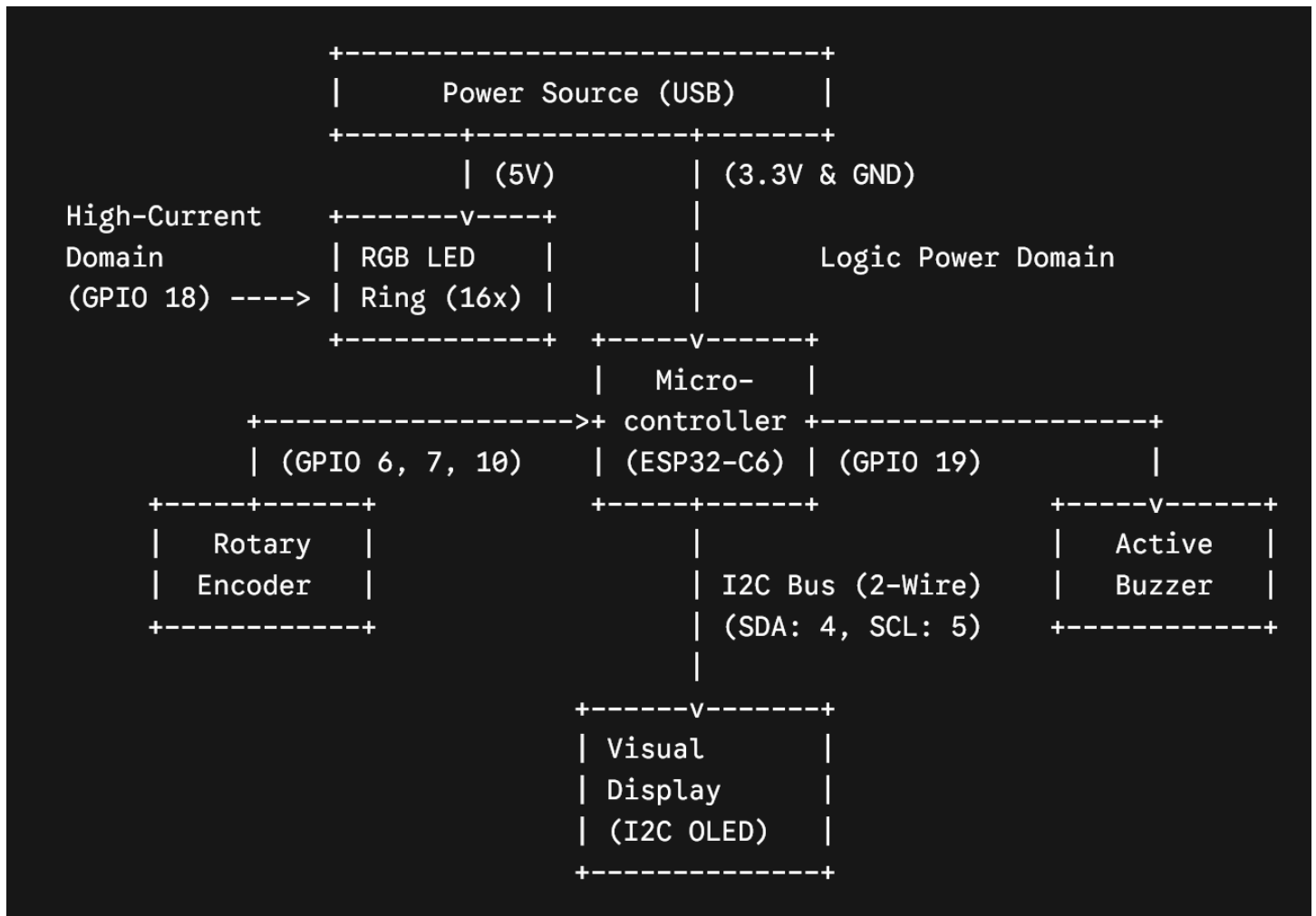
Hello! This section is designed to assist in the case of components variance. In simpler terms, did you get a component that has the same functions but the connections are different?

Some examples include:

- Your OLED Display has 4 pins instead of 7 pins? It turns out you got an [I2C Version!](#)
- Your buzzer module is either 2 pins or 3 pins

For Project Safecracker, we are now working with an I2C OLED Display. Code changes should be minor and your wiring will be simplified in the process.

Circuit Block Diagram:



C. Connections (I2C Version)

Power Rule Reminder: The LED Ring draws a lot of power. It must be connected to the **5V** pin (sometimes labeled VBUS) on the ESP32-C6 so it pulls power directly from the USB plug. Everything else safely uses the **3.3V** pin.

Component	ESP32-C6 Pin	Description
Encoder CLK	GPIO 6	Reads the first rotation signal (Phase A).
Encoder DT	GPIO 7	Reads the second rotation signal (Phase B).
Encoder SW	GPIO 10	The push-button built into the dial.
Encoder VCC	3.3V	Logic power for the dial.
Encoder GND	GND	Common ground connection.
OLED SDA	GPIO 4	The I2C Data line for sending text to the screen.
OLED SCL	GPIO 5	The I2C Clock line to keep the screen timed correctly.
OLED VCC	3.3V	Logic power to turn the screen on.
OLED GND	GND	Common ground connection.
LED Ring DI	GPIO 18	Sends the color data to the LED ring.
LED Ring 5V	5V / VBUS	High-Power Rail directly from the USB port.
LED Ring GND	GND	Common ground connection.
Buzzer I/O	GPIO 19	Triggers the beep sound. <i>(If using a 3-pin buzzer, connect its VCC to 3.3V and GND to GND).</i>

Code

```
1. #include <Arduino.h>
2. #include <Wire.h>
3. #include <U8g2lib.h>
4. #include <Adafruit_NeoPixel.h>
5.
6. // --- I2C OLED Pin Definitions ---
7. #define I2C_SDA 4
8. #define I2C_SCL 5
9.
10. // --- Hardware Pin Definitions ---
11. #define ENCODER_CLK 6
12. #define ENCODER_DT 7
13. #define ENCODER_SW 10
14.
15. #define LED_RING_PIN 18
16. #define BUZZER_PIN 19
17. #define NUM_LEDS 16
18.
19. // --- Object Initialization ---
20. // Using the I2C version of the SSD1306 screen
21. U8G2_SSD1306_128X64_NONAME_F_HW_I2C oled(U8G2_R0, U8X8_PIN_NONE);
22. Adafruit_NeoPixel ring(NUM_LEDS, LED_RING_PIN, NEO_GRB + NEO_KHZ800);
23.
24. // --- Security Vault Modes ---
25. enum SystemState { LOCKED, SUCCESS, ALARM };
26. SystemState currentState = LOCKED;
27.
28. const int PASSCODE[3] = {9, 1, 15}; // The secret 3-digit combination
29. int userInput[3] = {0, 0, 0};
30. int currentDigitIndex = 0;
31. int failedAttempts = 0;
32.
33. // --- Dial Tracking Variables ---
34. volatile int encoderValue = 0;
35. volatile uint8_t previousPhase = 0;
36. int lastEncoderValue = 0;
37.
38. // --- Advanced Hardware Interrupt (Ignores bouncy metal contacts) ---
39. void IRAM_ATTR readEncoder() {
40.     uint8_t currentA = digitalRead(ENCODER_CLK);
41.     uint8_t currentB = digitalRead(ENCODER_DT);
42.
43.     uint8_t currentPhase = (currentA << 1) | currentB;
44.     uint8_t movementState = (previousPhase << 2) | currentPhase;
45.
46.     // Exact clockwise physical sequence
47.     if (movementState == 0b1101 || movementState == 0b0100 || movementState == 0b0010 || movementState ==
0b1011) {
48.         encoderValue++;
49.         if (encoderValue > 15) encoderValue = 0;
50.     }
51.     // Exact counter-clockwise physical sequence
52.     else if (movementState == 0b1110 || movementState == 0b0111 || movementState == 0b0001 || movementState ==
0b1000) {
53.         encoderValue--;
54.         if (encoderValue < 0) encoderValue = 15;
55.     }
56.     previousPhase = currentPhase;
57. }
58.
59.
60. // --- Helper Functions ---
61. void setRingColor(uint8_t r, uint8_t g, uint8_t b) {
62.     for(int i = 0; i < NUM_LEDS; i++) {
63.         ring.setPixelColor(i, ring.Color(r, g, b));
64.     }
65.     ring.show();
66. }
67.
```

```

68. void soundChirp() {
69.   digitalWrite(BUZZER_PIN, HIGH);
70.   delay(40);
71.   digitalWrite(BUZZER_PIN, LOW);
72. }
73.
74. void setup() {
75.   Serial.begin(115200);
76.
77.   // Setup Hardware Pins
78.   pinMode(BUZZER_PIN, OUTPUT);
79.   pinMode(ENCODER_SW, INPUT_PULLUP);
80.   pinMode(ENCODER_CLK, INPUT_PULLUP);
81.   pinMode(ENCODER_DT, INPUT_PULLUP);
82.
83.   // Attach the interrupts to read the dial instantly in the background
84.   attachInterrupt(digitalPinToInterrupt(ENCODER_CLK), readEncoder, CHANGE);
85.   attachInterrupt(digitalPinToInterrupt(ENCODER_DT), readEncoder, CHANGE);
86.
87.   // Start the I2C Bus and the OLED Screen
88.   Wire.begin(I2C_SDA, I2C_SCL);
89.   oled.begin();
90.
91.   // Start the LED Ring (Limited to 40 brightness for safety!)
92.   ring.begin();
93.   ring.setBrightness(40);
94.   setRingColor(0, 0, 0);
95. }
96.
97. void loop() {
98.   unsigned long currentMillis = millis();
99.
100.  switch (currentState) {
101.
102.    // =====
103.    // MODE 1: LOCKED (Waiting for user input)
104.    // =====
105.    case LOCKED: {
106.      userInput[currentDigitIndex] = encoderValue;
107.
108.      // 1. Draw the yellow dot on the LED ring to match the dial
109.      ring.clear();
110.      ring.setPixelColor(encoderValue, ring.Color(255, 200, 0));
111.      ring.show();
112.
113.      // 2. Play a click sound if the dial moved
114.      if (encoderValue != lastEncoderValue) {
115.        soundChirp();
116.        lastEncoderValue = encoderValue;
117.      }
118.
119.      // 3. Did the user press the dial button down?
120.      if (digitalRead(ENCODER_SW) == LOW) {
121.        delay(50); // Ignore button bounce
122.        while(digitalRead(ENCODER_SW) == LOW); // Wait for them to let go
123.
124.        soundChirp(); delay(50); soundChirp();
125.
126.        currentDigitIndex++;
127.        encoderValue = 0; // Reset dial back to 0 for the next slot
128.
129.        // Check the passcode if all 3 numbers are entered
130.        if (currentDigitIndex >= 3) {
131.          if (userInput[0] == PASSCODE[0] && userInput[1] == PASSCODE[1] && userInput[2] == PASSCODE[2]) {
132.            currentState = SUCCESS; // Password correct!
133.          } else {
134.            failedAttempts++;
135.            setRingColor(255, 0, 0); // Flash Red
136.            digitalWrite(BUZZER_PIN, HIGH);
137.            delay(500);
138.            digitalWrite(BUZZER_PIN, LOW);
139.

```

```

140.         if (failedAttempts >= 3) {
141.             currentState = ALARM; // Too many failures!
142.         } else {
143.             currentDigitIndex = 0; // Try again
144.             setRingColor(0, 0, 0);
145.         }
146.     }
147. }
148. }
149.
150. // 4. Draw the Vault Interface on the OLED Screen
151. oled.clearBuffer();
152. oled.setFont(u8g2_font_6x10_tf);
153. oled.setCursor(18, 12); oled.print("SECURE TERMINAL");
154. oled.drawHLine(0, 16, 128);
155.
156. oled.setFont(u8g2_font_logisoso22_tr);
157.
158. // Draw Slot 1
159. if (currentDigitIndex > 0) oled.drawStr(20, 48, "*");
160. else if (currentDigitIndex == 0) { oled.setCursor(20, 48); oled.print(userInput[0]); }
161. else oled.drawStr(20, 48, "-");
162.
163. // Draw Slot 2
164. if (currentDigitIndex > 1) oled.drawStr(58, 48, "*");
165. else if (currentDigitIndex == 1) { oled.setCursor(58, 48); oled.print(userInput[1]); }
166. else oled.drawStr(58, 48, "-");
167.
168. // Draw Slot 3
169. if (currentDigitIndex > 2) oled.drawStr(96, 48, "*");
170. else if (currentDigitIndex == 2) { oled.setCursor(96, 48); oled.print(userInput[2]); }
171. else oled.drawStr(96, 48, "-");
172.
173. // Draw the line under the active slot
174. if (currentDigitIndex == 0) oled.drawHLine(18, 52, 20);
175. if (currentDigitIndex == 1) oled.drawHLine(56, 52, 20);
176. if (currentDigitIndex == 2) oled.drawHLine(94, 52, 20);
177.
178. oled.sendBuffer();
179.
180. // Send data to the computer for debugging
181. Serial.print("State:"); Serial.print(currentState); Serial.print(",");
182. Serial.print("Encoder:"); Serial.print(encoderValue); Serial.print(",");
183. Serial.print("Slot:"); Serial.print(currentDigitIndex); Serial.print(",");
184. Serial.print("Failures:"); Serial.println(failedAttempts);
185. break;
186. }
187.
188. // =====
189. // MODE 2: SUCCESS (Vault Open)
190. // =====
191. case SUCCESS: {
192.     setRingColor(0, 255, 0); // Solid Green Ring
193.
194.     oled.clearBuffer();
195.     oled.setFont(u8g2_font_9x15B_tf);
196.     oled.setCursor(22, 38);
197.     oled.print("UNLOCKED");
198.     oled.sendBuffer();
199.
200.     Serial.print("State:"); Serial.println("SUCCESS_UNLOCKED");
201.     break;
202. }
203.
204. // =====
205. // MODE 3: ALARM (Lockdown!)
206. // =====
207. case ALARM: {
208.     // Flash the lights and sound the siren every 200 milliseconds
209.     if ((currentMillis / 200) % 2 == 0) {
210.         setRingColor(255, 0, 0);
211.         digitalWrite(BUZZER_PIN, HIGH);

```

```
212.     } else {
213.         setRingColor(0, 0, 0);
214.         digitalWrite(BUZZER_PIN, LOW);
215.     }
216.
217.     oled.clearBuffer();
218.     oled.setFont(u8g2_font_9x15B_tf);
219.     oled.setCursor(30, 38);
220.     oled.print("LOCKDOWN");
221.     oled.sendBuffer();
222.
223.     Serial.print("State:"); Serial.println("ALARM_LOCKDOWN");
224.     break;
225. }
226. }
227. }
228.
```